

CARE4SUCCESS

Guide de Prompts pour le Développement IA

À l'attention du Chef de Projet Technique :

Ce document est votre feuille de route opérationnelle. Il traduit la vision métier de la plateforme de soutien scolaire CARE4SUCCESS en instructions techniques précises (prompts) destinées à une IA de génération de code (ex: GitHub Copilot, Cursor, ChatGPT, Claude).

Règles d'or pour l'utilisation de ces prompts :

- **Séquencement** : Ne donnez jamais deux modules à coder en même temps. Attendez que l'IA valide et corrige le module N avant de passer au module N+1.
- **Contexte persistant** : Si vous ouvrez une nouvelle session de chat avec l'IA, rappelez-lui toujours la stack technique (React 18 + Vite 5 + TypeScript strict + Tailwind CSS 4 + API Express 5 + MySQL) et le schéma de base de données existant (voir section ci-dessous).
- **Itération** : Ces prompts génèrent la structure et la logique de base. Vous devrez ensuite demander à l'IA d'ajuster le design (Tailwind + Shadcn UI) selon votre maquette exacte.



Stack Technique & Base de Données

Cette section est la référence technique centrale du projet CARE4SUCCESS. Fournissez-la à l'IA au début de chaque nouvelle session pour garantir la cohérence du code généré.



Frontend

FRAMEWORK & BUILD

- React 18 + Vite 5

UI & STYLES

- Tailwind CSS 4

- `TypeScript` strict
- Composants : `Shadcn UI`
- Routing : `React Router 6` (HashRouter)
- Icônes : `Lucide React`

DATA FETCHING

- `React Query 5` (TanStack Query)

Backend

SERVEUR

- `Express 5` sur `Node.js`
- `mysql2/promise` (driver MySQL)
- `dotenv` / `cors`

BASE DE DONNÉES

- **MySQL** (base principale)
- Scripts Supabase optionnels (migrations/seed Postgres)

Outils & Scripts

- `ESLint 9` + `TypeScript 5`
- `concurrently` (tâches parallèles front/back)
- Bundling : `Vite`
- Scripts npm : `test` / `lint`

Base de Données MySQL — Tables existantes

Scripts d'initialisation :

- Créer la base : `mysql -u root -p < server/schema.sql`
- Préremplir la base : `mysql -u root -p < server/seed.sql`
- `requests` — demandes de cours
- `teacher_feedback` — évaluations enseignants
- `assignments` — devoirs
- `parent_overviews` — vue d'ensemble parent
- `sessions` — sessions de cours
- `teacher_applications` — candidatures enseignants
- `student_progress_points` — points de progression élève

- `parent_invoices` — factures parent
- `quizzes` — quiz / QCM
- `courses` — cours
- `quiz_questions` — questions de quiz
- `course_lessons` — leçons par cours
- `quiz_attempts` — tentatives de quiz
- `course_enrollments` — inscriptions aux cours



PHASE 1 : Lancement (Avant fin du mois)

Cette phase se concentre exclusivement sur le MVP (Minimum Viable Product). L'objectif est d'avoir une plateforme fonctionnelle permettant l'inscription, la gestion des cours, le suivi et la facturation de base.

MODULE 1 : Gestion des profils et Authentification

La fondation du système. L'authentification est gérée côté serveur Express + JWT. Nous devons structurer les tables de profils et leurs relations (clés étrangères) directement en MySQL.

PROMPT À COPIER-COLLER

Agis en tant que développeur Full-Stack Senior expert en React 18 + Vite 5, TypeScript strict, Tailwind CSS 4, Shadcn UI, React Router 6 (HashRouter), React Query 5 côté frontend, et Express 5 + Node.js + mysql2/promise côté backend.

CONTEXTE DU PROJET :

Je développe "CARE4SUCCESS", une plateforme de soutien scolaire. Le système repose sur 5 rôles distincts : Admin, Parent, Élève, Conseiller, et Tuteur. La base de données est MySQL. Les scripts d'init se trouvent dans `server/schema.sql` et `server/seed.sql`.

OBJECTIF DE CETTE ÉTAPE :

Créer le système d'authentification et la gestion des profils utilisateurs.

TABLES MYSQL CONCERNÉES :

- ⚠ Ces tables sont à créer (pas encore dans le schéma existant) :
- ``users`` – table centrale avec colonnes : `id`, `email`, `password_hash`, `role`

```
(ENUM: admin, parent, student, counselor, tutor), created_at
- `parent_child` – liaison Parent ↔ Élève (parent_id, child_id)
- `student_teacher` – liaison Élève ↔ Enseignant (student_id, teacher_id)
```

FONCTIONNALITÉS À CODER :

1. Authentification JWT : Routes Express POST /auth/register et POST /auth/login. Génération et vérification de tokens JWT (jsonwebtoken). Middleware d'authentification pour protéger les routes privées.
2. Création des tables MySQL via migration SQL : table `users` avec colonne `role`, tables de liaison `parent\_child` et `student\_teacher`.
3. Logique de liaison :
 - Un Parent peut être lié à plusieurs Enfants (Élèves) → table `parent\_child`.
 - Un Élève peut être lié à plusieurs Enseignants → table `student\_teacher`.
4. Interface UI (React + Shadcn UI) : Formulaires de création et de modification de profil dynamiques selon le rôle sélectionné. Utilise React Query 5 pour les appels API.

BONNES PRATIQUES ET CONTRAINTES :

- Utilise TypeScript strict avec des interfaces/types pour chaque rôle utilisateur.
- Hache les mots de passe avec bcrypt côté Express avant insertion MySQL.
- Crée des composants de formulaire réutilisables (Input, Select) stylisés avec Tailwind CSS 4 et Shadcn UI.
- Utilise HashRouter (React Router 6) pour la navigation.
- Les requêtes front vers le backend utilisent React Query 5 (useQuery / useMutation).

Génère le SQL MySQL pour les nouvelles tables, les routes Express + middleware JWT, les hooks React Query personnalisés (ex: useProfile), et les composants React.

MODULE 2 : Dashboard Parent

Le centre de contrôle pour les clients payants. Il doit être rassurant, clair et agréger les données provenant des élèves et des professeurs.

PROMPT À COPIER-COLLER

Agis en tant que développeur Full-Stack Senior (React 18 + Vite 5, TypeScript strict, Tailwind CSS 4, Shadcn UI, React Query 5, Express 5, mysql2/promise).

CONTEXTE DU PROJET :

Plateforme CARE4SUCCESS. Le module de gestion des profils est terminé. Nous passons à l'interface des parents.

Base de données : MySQL. Scripts : server/schema.sql / server/seed.sql.

OBJECTIF DE CETTE ÉTAPE :

Créer le "Dashboard Parent", une vue centralisée permettant au parent de suivre la scolarité de ses enfants.

TABLES MYSQL CONCERNÉES :

- `parent\_overviews` – données agrégées de la vue parent
 - `parent\_invoices` – factures parent (statut payé/non payé)
 - `sessions` – sessions de cours (planning, historique)
 - `assignments` – devoirs assignés aux enfants
 - `courses` – détail des cours suivis
- (Lecture via JOIN sur `parent\_child` → `users` pour lier parent ↔ enfants)

FONCTIONNALITÉS À CODER (UI et routes API Express) :

1. Vue "Mes Enfants" : Liste des enfants liés au parent (JOIN parent_child + users).
2. Vue "Équipe Pédagogique" : Liste des enseignants assignés à ses enfants (JOIN student_teacher + users).
3. Vue "Planning & Historique" : Calendrier des sessions à venir et liste des sessions passées (table `sessions`).
4. Vue "Suivi Académique" : Commentaires des enseignants post-session (teacher_feedback via sessions), devoirs assignés (assignments).
5. Vue "Finances" : Liste des factures (parent_invoices) avec badge de statut (Payé / Non Payé).

RELATIONS & DÉPENDANCES :

- Routes Express GET /api/parent/:id/dashboard effectuant des requêtes JOIN MySQL complexes.

BONNES PRATIQUES ET CONTRAINTES :

- Crée un layout dashboard avec sidebar de navigation (Shadcn UI + Tailwind CSS 4).
- Utilise des composants Card Shadcn UI pour les résumés (ex: Prochain cours, Dernière note).
- Gère les états de chargement (Skeletons Shadcn) et les erreurs via React Query 5.
- Vérifie côté Express que le parent authentifié (JWT) n'accède qu'aux données de ses propres enfants.

Génère les routes Express + requêtes MySQL avec JOINS, les hooks React Query, et les composants d'affichage.

MODULE 3 : Dashboard Élève

L'interface quotidienne de l'utilisateur final. Elle doit être orientée sur l'action (faire ses devoirs, voir son planning).

PROMPT À COPIER-COLLER

Agis en tant que développeur Full-Stack Senior (React 18 + Vite 5, TypeScript strict, Tailwind CSS 4, Shadcn UI, React Query 5, Express 5, mysql2/promise).

CONTEXTE DU PROJET :

Plateforme CARE4SUCCESS. Nous développons maintenant l'espace dédié aux élèves.

Base de données : MySQL. Scripts : server/schema.sql / server/seed.sql.

OBJECTIF DE CETTE ÉTAPE :

Créer le "Dashboard Élève", axé sur l'organisation et l'évaluation.

TABLES MYSQL CONCERNÉES :

- `student\_progress\_points` – points/notes de progression de l'élève
- `assignments` – devoirs à faire / soumis
- `quizzes` – quiz/QCM disponibles
- `quiz\_questions` – questions de chaque quiz
- `quiz\_attempts` – tentatives et résultats de l'élève
- `sessions` – sessions de cours (planning + historique)
- `courses` – cours suivis
- `course\_lessons` – leçons de chaque cours
- `course\_enrollments` – inscriptions de l'élève aux cours

⚠ Table à créer : `grade\_disputes` (id, student_id, session_id, reason, status, created_at)

FONCTIONNALITÉS À CODER :

1. Planning interactif : Affichage des sessions à venir (table `sessions`).
2. Annuaire : Liste de ses enseignants (student_teacher + users).
3. Espace Devoirs & QCM : Interface pour voir les devoirs (assignments), répondre à des QCM (quizzes + quiz_questions), et enregistrer les tentatives (quiz_attempts).
4. Espace Notes / Progression : Historique des points (student_progress_points).
5. Système de contestation : Formulaire "Contester une note" → INSERT dans `grade\_disputes`.
6. Historique des cours : Sessions passées (sessions + courses).

BONNES PRATIQUES ET CONTRAINTES :

- L'UI doit être épurée et accessible (Tailwind CSS 4 + Shadcn UI).
- Créer un composant réutilisable DataTable (Shadcn UI) pour l'historique et

les notes.

- Utilise React Query 5 (useQuery / useMutation) pour toutes les interactions API.
- Valide les formulaires avec React Hook Form + Zod.

Génère les routes Express + requêtes MySQL, les types TypeScript pour devoirs/quiz/progression, et les composants React.

MODULE 4 : Système de cours

Le cœur opérationnel de la plateforme. La précision des horodatages (timestamps) est cruciale ici pour la facturation ultérieure.

PROMPT À COPIER-COLLER

Agis en tant que développeur Full-Stack Senior (React 18 + Vite 5, TypeScript strict, Tailwind CSS 4, Shadcn UI, React Query 5, Express 5, mysql2/promise).

CONTEXTE DU PROJET :

Plateforme CARE4SUCCESS. Nous devons maintenant gérer le déroulement d'une session de cours.

Base de données : MySQL. Scripts : server/schema.sql / server/seed.sql.

OBJECTIF DE CETTE ÉTAPE :

Créer le "Système de cours" utilisé par les enseignants pendant leurs sessions avec les élèves.

TABLES MYSQL CONCERNÉES :

- `sessions` – sessions de cours (actual_start_time, actual_end_time, report_text, understanding_score, is_paid)
- `courses` – cours associés à chaque session
- `course\_lessons` – leçons dispensées pendant la session
- `assignments` – devoirs assignés en fin de session
- `teacher\_feedback` – évaluation/rapport post-session du professeur

FONCTIONNALITÉS À CODER :

1. Tracking de temps : Boutons "Check-in" (début) et "Check-out" (fin) → PATCH /api/sessions/:id avec timestamp serveur (NE PAS utiliser l'heure du navigateur client, utiliser NOW() MySQL côté Express).
2. Rapport de cours : Formulaire post-cours obligatoire pour le professeur → UPDATE de `sessions` + INSERT dans `teacher\_feedback`.
3. Évaluation : Champ "Note de compréhension" (understanding_score) dans

`sessions`.

4. Devoirs : Possibilité d'assigner un devoir à la fin du cours → INSERT dans `assignments`.

5. Automatisation (cours en ligne) : Logique pour enregistrer automatiquement start/end depuis un lien visio.

BONNES PRATIQUES ET CONTRAINTES :

- Les timestamps sont toujours générés côté serveur Express (MySQL NOW()) pour garantir l'intégrité – ne jamais utiliser new Date() côté client.
- Le formulaire de rapport utilise React Hook Form + Zod pour la validation.
- Utilise React Query 5 (useMutation) pour les actions check-in / check-out / soumission rapport.
- Ajoute un flag `is\_paid` (BOOLEAN) dans `sessions` pour éviter les doubles paiements ultérieurs.

Génère la logique de tracking (routes Express + MySQL), et l'interface React du professeur pour gérer son cours.

MODULE 5 : Module Conseiller

Le conseiller est le chef d'orchestre. Il a besoin d'une vue globale et d'outils d'action rapide (matching manuel).

PROMPT À COPIER-COLLER

Agis en tant que développeur Full-Stack Senior (React 18 + Vite 5, TypeScript strict, Tailwind CSS 4, Shadcn UI, React Query 5, Express 5, mysql2/promise).

CONTEXTE DU PROJET :

Plateforme CARE4SUCCESS. Le rôle du "Conseiller" est d'accompagner les familles et de gérer les problèmes.

Base de données : MySQL. Scripts : server/schema.sql / server/seed.sql.

OBJECTIF DE CETTE ÉTAPE :

Créer le "Dashboard Conseiller", un outil de supervision et de gestion.

TABLES MYSQL CONCERNÉES :

- `requests` – demandes de cours en attente de traitement
 - `sessions` – historique global des cours
 - `student\_progress\_points` – progression des élèves supervisés
 - `parent\_overviews` – vue d'ensemble des familles
- (Lecture étendue via JOIN sur `users`, `parent\_child`, `student\_teacher`)

FONCTIONNALITÉS À CODER :

1. Vues globales avec filtres : Tableaux listant parents, élèves et enseignants de son secteur. Filtres multicritères (matière, niveau, statut).
2. Suivi de progression : Courbe de progression d'un élève (student_progress_points triés par date).
3. Historique global : Accès à toutes les sessions de cours.
4. Gestion des alertes : Vue "Problèmes remontés" (grade_disputes, absences, retards depuis sessions).
5. Matching Manuel : Interface permettant au conseiller de sélectionner un élève + un enseignant disponible → INSERT dans `student\_teacher`, et mise à jour du statut dans `requests`.

BONNES PRATIQUES ET CONTRAINTES :

- Les routes Express appliquent un middleware de rôle : seul le rôle `counselor` (ou `admin`) peut accéder à ces endpoints.
- Implémente la pagination côté MySQL (LIMIT / OFFSET) pour les grands volumes de données.
- Pour le matching manuel, crée un composant UI à double sélection (Select Shadcn) très intuitif.
- Utilise React Query 5 pour la gestion du cache et des rechargements automatiques.

Génère les vues de supervision, les routes Express paginées, et la logique complète du matching manuel.

MODULE 6 : Module Tuteur (Évaluation Enseignants)

Le tuteur est responsable du recrutement et de la qualité des enseignants. Ce module est un mini-ATS (Applicant Tracking System).

PROMPT À COPIER-COLLER

Agis en tant que développeur Full-Stack Senior (React 18 + Vite 5, TypeScript strict, Tailwind CSS 4, Shadcn UI, React Query 5, Express 5, mysql2/promise).

CONTEXTE DU PROJET :

Plateforme CARE4SUCCESS. Le rôle du "Tuteur" est d'évaluer et de classifier les candidats enseignants.

Base de données : MySQL. Scripts : server/schema.sql / server/seed.sql.

OBJECTIF DE CETTE ÉTAPE :

Créer le "Module Tuteur" pour la gestion du recrutement pédagogique.

TABLES MYSQL CONCERNÉES :

- ``teacher_applications`` – candidatures des enseignants (statut : pending, interview, validated, rejected)
- ``teacher_feedback`` – évaluations et rapports post-entretien du tuteur

FONCTIONNALITÉS À CODER :

1. Fiche Candidat : Création et visualisation des profils enseignants postulants (INSERT/SELECT dans ``teacher_applications``).
2. Planification : Outil pour planifier un entretien → UPDATE du statut dans ``teacher_applications`` + champ ``interview_date``.
3. Rapport d'évaluation : Formulaire détaillé post-entretien → INSERT dans ``teacher_feedback`` lié à l'application.
4. Classification : Système de tags/catégories (Matière + Niveau) stocké en JSON dans ``teacher_applications`` ou via colonnes dédiées. Affichage sous forme de badges Shadcn UI.

BONNES PRATIQUES ET CONTRAINTES :

- Utilise TypeScript strict avec des interfaces pour ``TeacherApplication`` et ``TeacherFeedback``.
- Valide les formulaires d'évaluation avec React Hook Form + Zod.
- L'interface de classification utilise des badges visuels Shadcn (Badge component) pour identifier rapidement les compétences.
- Middleware Express : seul le rôle ``tutor`` (ou ``admin``) peut créer/modifier des évaluations.

Génère les routes Express + requêtes MySQL, les types TypeScript, et les composants React pour le flux de recrutement.

MODULE 7 : Facturation simple

Dernière étape du MVP. Sans cela, la plateforme ne génère pas de revenus. On reste sur du déclaratif et de la génération de documents pour l'instant.

PROMPT À COPIER-COLLER

Agis en tant que développeur Full-Stack Senior (React 18 + Vite 5, TypeScript strict, Tailwind CSS 4, Shadcn UI, React Query 5, Express 5, mysql2/promise).

CONTEXTE DU PROJET :

Plateforme CARE4SUCCESS. Nous finalisons la Phase 1 (MVP) avec la gestion

financière de base.

Base de données : MySQL. Scripts : server/schema.sql / server/seed.sql.

OBJECTIF DE CETTE ÉTAPE :

Créer le système de "Facturation simple" pour les parents.

TABLES MYSQL CONCERNÉES :

- `parent\_invoices` – factures parent (id, parent_id, amount, status ENUM(paid, unpaid), period_start, period_end, created_at)
- `sessions` – sessions de cours (actual_start_time, actual_end_time, is_paid) pour calculer les heures facturées

FONCTIONNALITÉS À CODER :

1. Génération de facture : Route Express POST /api/invoices – calcul du montant mensuel en croisant `sessions` (durée = actual_end_time - actual_start_time) et le tarif horaire du parent → INSERT dans `parent\_invoices`.
2. Statut de paiement : Route PATCH /api/invoices/:id/status – toggle Payé/Non Payé, accessible uniquement par l'Admin.
3. Historique : Affichage dans le Dashboard Parent (Module 2, déjà préparé) – GET /api/parent/:id/invoices.

BONNES PRATIQUES ET CONTRAINTES :

- Le calcul des heures se fait côté serveur Express (MySQL TIMESTAMPDIF) – jamais côté client.
- Utilise une librairie comme `@react-pdf/renderer` ou `jspdf` pour permettre le téléchargement PDF de la facture côté client.
- Sécurise la modification du statut "Payé" via un middleware de rôle Express (seul admin peut modifier, le parent ne peut que lire).
- Marque les sessions concernées (`is\_paid = TRUE` dans `sessions`) pour éviter la double facturation.

Génère la logique de calcul MySQL, les routes Express, et le composant React de génération/affichage des factures.



PHASE 2 : Automatisation (3 mois après lancement)

La plateforme tourne. L'objectif est maintenant de réduire la charge de travail manuel (matching, RH, paiements sortants) pour permettre le passage à l'échelle (scaling).

MODULE 8 : Matching automatique enseignant/élève

On remplace l'action manuelle du conseiller par un algorithme de recommandation basé sur des critères stricts.

PROMPT À COPIER-COLLER

Agis en tant que développeur Full-Stack Senior (React 18 + Vite 5, TypeScript strict, Tailwind CSS 4, Shadcn UI, React Query 5, Express 5, mysql2/promise).

CONTEXTE DU PROJET :

Plateforme CARE4SUCCESS. Phase 2. Nous voulons automatiser l'attribution des professeurs aux élèves.

Base de données : MySQL. Scripts : server/schema.sql / server/seed.sql.

OBJECTIF DE CETTE ÉTAPE :

Créer un algorithme de "Matching automatique" et son interface de validation.

TABLES MYSQL CONCERNÉES :

- `requests` – demandes de cours (matière, niveau, budget, localisation du parent)
- `teacher\_applications` – profils enseignants validés (compétences, tarif, localisation, niveaux enseignés)
- `sessions` – historique pour évaluer la disponibilité des enseignants (Action finale : INSERT dans `student\_teacher` après validation du match)

FONCTIONNALITÉS À CODER :

1. Algorithme de scoring : Logique Express (TypeScript) qui analyse et croise :
 - Matière demandée (requests) vs compétences du prof (teacher_applications) – coefficient 3.
 - Localisation/quartier – coefficient 2.
 - Budget parent vs tarif prof – coefficient 2.
 - Niveau élève vs niveaux enseignés par le prof – coefficient 3.
2. Proposition automatique : Route GET /api/matching/:request_id → retourne un "Top 3" des meilleurs profs avec score de compatibilité.
3. Interface de validation : Le conseiller ou parent voit les suggestions (% de compatibilité) et valide → INSERT dans `student\_teacher`, UPDATE statut `requests`.

BONNES PRATIQUES ET CONTRAINTES :

- Écris la logique de scoring comme une fonction TypeScript pure dans un fichier dédié `server/services/matching.service.ts` pour faciliter les tests unitaires.
- Utilise des requêtes MySQL paramétrées avec mysql2/promise pour récupérer les candidats éligibles.
- L'UI affiche les "Matches recommandés" avec des jauges de compatibilité

(SVG ou Tailwind CSS).

- Utilise React Query 5 pour déclencher le calcul et afficher les résultats en temps réel.

Génère le service de scoring TypeScript, la route Express, et le composant React d'affichage des recommandations.

MODULE 9 : Module RH Admin

Structuration de l'entreprise. Il faut gérer les statuts légaux et les documents des collaborateurs.

PROMPT À COPIER-COLLER

Agis en tant que développeur Full-Stack Senior (React 18 + Vite 5, TypeScript strict, Tailwind CSS 4, Shadcn UI, React Query 5, Express 5, mysql2/promise).

CONTEXTE DU PROJET :

Plateforme CARE4SUCCESS. Phase 2. L'administration a besoin d'outils de gestion interne.

Base de données : MySQL. Scripts : server/schema.sql / server/seed.sql.

OBJECTIF DE CETTE ÉTAPE :

Créer le "Module RH Admin" pour gérer le staff (Employés, Consultants, Tuteurs, Conseillers).

TABLES MYSQL CONCERNÉES :

- `teacher\_applications` – base des dossiers enseignants (statut, informations contractuelles)

- `teacher\_feedback` – historique des évaluations de performance

⚠️ Nouvelles tables RH à créer :

- `hr\_records` (id, user_id, contract_type ENUM(employee, consultant), contract_file_path, created_at)

- `payslips` (id, user_id, month, year, file_path, distributed_at)

FONCTIONNALITÉS À CODER :

1. Gestion des statuts : Différencier employés (salariés) et consultants (freelances) via `hr\_records.contract\_type`.

2. Gestion documentaire : Upload et stockage sécurisé des contrats → fichier sauvegardé sur le serveur Node.js (multer + chemin stocké dans `hr\_records.contract\_file\_path`). Accès protégé par middleware de rôle.

3. Fiches de paie : Espace pour uploader et distribuer les fiches de paie

mensuelles → INSERT dans `payslips`.

4. Suivi des paiements internes : Tableau de bord des rémunérations dues.

BONNES PRATIQUES ET CONTRAINTES :

- Utilise `multer` côté Express pour l'upload de fichiers (contrats, fiches de paie).
- Configure des middlewares Express stricts : seul l'Admin et l'employé concerné peuvent accéder à leurs documents.
- L'interface permet un upload par glisser-déposer (composant Shadcn UI + react-dropzone).
- Génère le SQL MySQL pour les nouvelles tables `hr\_records` et `payslips`.

Génère le SQL des nouvelles tables, les routes Express avec multer, et l'interface RH complète.

MODULE 10 : Module paiement enseignants

Automatisation des flux financiers sortants pour éviter les erreurs de calcul à la fin du mois.

PROMPT À COPIER-COLLER

Agis en tant que développeur Full-Stack Senior (React 18 + Vite 5, TypeScript strict, Tailwind CSS 4, Shadcn UI, React Query 5, Express 5, mysql2/promise).

CONTEXTE DU PROJET :

Plateforme CARE4SUCCESS. Phase 2. Suite au module RH, nous devons automatiser la paie des intervenants.

Base de données : MySQL. Scripts : server/schema.sql / server/seed.sql.

OBJECTIF DE CETTE ÉTAPE :

Créer le "Module de paiement automatisé" pour les tuteurs, enseignants et conseillers.

TABLES MYSQL CONCERNÉES :

- `parent\_invoices` – revenus encaissés (base de calcul pour les commissions)
- `sessions` – heures travaillées par session (actual_start_time, actual_end_time, is_paid)

⚠ Nouvelles tables paiements à créer :

- `payouts` (id, user_id, period_start, period_end, hours_total, amount_due, status ENUM(pending, paid), created_at)

- (Optionnel) ``payout_sessions`` – table de liaison payout ↔ sessions couvertes

FONCTIONNALITÉS À CODER :

1. Moteur de calcul : Script Node.js (`server/services/payroll.service.ts`) calculant la rémunération de chaque intervenant à partir des sessions validées (`is_paid = FALSE`) et du taux horaire/commission configuré.
2. Génération de relevés : Création automatique d'un enregistrement ``payouts`` par intervenant + marquage des sessions couvertes (`is_paid = TRUE`).
3. Déclenchement mensuel : Exposer une route Express POST `/api/payroll/run` protégée admin, appellable via un cron job externe (`node-cron` ou service tiers).
4. Statut de virement : Interface Admin pour marquer les lots ``payouts`` comme `status = 'paid'`.

BONNES PRATIQUES ET CONTRAINTES :

- Utilise des transactions MySQL (`BEGIN / COMMIT / ROLLBACK` avec `mysql2/promise`) pour garantir l'atomicité : calcul + marquage sessions + création payout en une seule transaction.
- Les sessions déjà payées (`is_paid = TRUE`) sont exclues du calcul pour éviter les doubles paiements.
- Génère le SQL MySQL pour les nouvelles tables ``payouts`` et ``payout_sessions``.
- Utilise React Query 5 pour déclencher l'action et afficher le tableau de bord financier Admin.

Génère le SQL des nouvelles tables, le service de calcul TypeScript, la route Express, et le tableau de bord financier Admin.

PHASE 3 : Vision Long Terme (Innovation)

Ces fonctionnalités transformeront CARE4SUCCESS d'une simple plateforme de gestion en un véritable écosystème éducatif innovant et compétitif.

MODULE 11 : Olympiades (Gamification & Compétition)

Création d'événements pour stimuler l'engagement des élèves et créer une communauté.

 PROMPT À COPIER-COLLER

Agis en tant que développeur Full-Stack Senior (React 18 + Vite 5, TypeScript strict, Tailwind CSS 4, Shadcn UI, React Query 5, Express 5, mysql2/promise).

CONTEXTE DU PROJET :

Plateforme CARE4SUCCESS. Phase 3 (Long terme). Nous introduisons de la gamification.

Base de données : MySQL. Scripts : server/schema.sql / server/seed.sql.

OBJECTIF DE CETTE ÉTAPE :

Créer le système "Olympiades", permettant d'organiser des compétitions (Maths, Sciences).

TABLES MYSQL CONCERNÉES :

- `quizzes` – épreuves/examens des Olympiades (title, subject, duration_seconds, status)
- `quiz\_questions` – questions de chaque épreuve
- `quiz\_attempts` – tentatives et résultats des élèves (score, completed_at, time_spent)
- `student\_progress\_points` – points gagnés lors des Olympiades

⚠️ Nouvelles tables à créer :

- `badges` (id, name, description, icon_url, created_at)
- `student\_badges` (id, student_id, badge_id, awarded_at)

FONCTIONNALITÉS À CODER :

1. Interface d'examen chronométrée : Affichage des questions (quiz_questions) avec compte à rebours synchronisé avec le serveur. À la fin → INSERT dans `quiz\_attempts` avec le score calculé.
2. Surveillance basique : Détection de sortie d'onglet (visibilitychange API) → flag `tab\_switches` dans `quiz\_attempts`.
3. Correction automatique : Questions fermées (QCM) corrigées côté serveur Express pour éviter la triche.
4. Classement (Leaderboard) : GET /api/olympiades/:quiz_id/leaderboard – SELECT students triés par score depuis `quiz\_attempts`. Actualisation via polling React Query (refetchInterval).
5. Récompenses : INSERT dans `student\_badges` pour les gagnants + mise à jour de `student\_progress\_points`.

BONNES PRATIQUES ET CONTRAINTES :

- Le chronomètre est synchronisé avec le serveur : Express stocke `started\_at` dans `quiz\_attempts` à l'ouverture de l'épreuve. Le frontend calcule le temps restant à partir de cette valeur serveur.
- La correction se fait UNIQUEMENT côté Express (ne jamais envoyer les bonnes réponses au client).
- Génère le SQL MySQL pour les nouvelles tables `badges` et `student\_badges`.
- Utilise React Query 5 (polling) pour le Leaderboard temps réel.

Génère l'interface d'examen chronométrée, la logique anti-triche, les routes Express, et le composant Leaderboard.

MODULE 12 : IA Pédagogique

L'intégration de l'Intelligence Artificielle pour offrir un tutorat personnalisé 24/7 et assister les professeurs.

PROMPT À COPIER-COLLER

Agis en tant que développeur Full-Stack Senior (React 18 + Vite 5, TypeScript strict, Tailwind CSS 4, Shadcn UI, React Query 5, Express 5, mysql2/promise) et expert en intégration IA (OpenAI API).

CONTEXTE DU PROJET :

Plateforme CARE4SUCCESS. Phase 3. C'est l'aboutissement technologique : l'intégration de l'IA.

Base de données : MySQL. Scripts : server/schema.sql / server/seed.sql.

OBJECTIF DE CETTE ÉTAPE :

Intégrer une "IA Pédagogique" au service de l'élève.

TABLES MYSQL CONCERNÉES :

- `student\_progress\_points` – historique des notes/scores de l'élève (source principale d'analyse)
- `quiz\_attempts` – résultats des QCM (lacunes par matière/compétence)
- `assignments` – devoirs complétés ou non (engagement de l'élève)
- `sessions` – rapports post-cours des enseignants (commentaires pédagogiques)

FONCTIONNALITÉS À CODER :

1. Analyse des lacunes : Route Express GET /api/ai/analysis/:student_id – requêtes MySQL sur student_progress_points, quiz_attempts et sessions pour identifier les matières/compétences faibles. Construction d'un résumé structuré envoyé à OpenAI.
2. Recommandation d'exercices : Génération dynamique d'exercices ciblés via l'API OpenAI (gpt-4o) en fonction des lacunes détectées. La clé API reste UNIQUEMENT côté serveur Express (.env).
3. Chatbot Éducatif : Route Express POST /api/ai/chat (streaming SSE) avec un System Prompt strict de tuteur socratique. Interface de chat côté React utilisant fetch + ReadableStream pour le streaming des réponses.

BONNES PRATIQUES ET CONTRAINTES :

- La clé OpenAI est dans le .env Express et n'est JAMAIS exposée côté client React.
- Implémente un "System Prompt" strict : le chatbot guide la réflexion (méthode socratique) sans donner directement les réponses aux devoirs. Injecte dans le prompt le contexte MySQL de l'élève (dernières notes, matières faibles).
- Utilise le streaming SSE (Server-Sent Events) depuis Express pour une UI fluide (les réponses s'affichent mot par mot).
- Utilise React Query 5 pour les appels d'analyse non-streaming, et fetch natif pour le streaming du chatbot.

Génère les routes Express communiquant avec OpenAI, le prompt système du tuteur socratique, et le composant React du Chatbot avec streaming.



Conseil final pour le Chef de Projet :

L'IA de coding est puissante mais n'a pas de mémoire infinie. Au fur et à mesure que le projet grandit, maintenez un fichier `SCHEMA.md` contenant la structure complète de votre base de données MySQL (toutes les tables, colonnes, types, clés étrangères). Fournissez ce fichier à l'IA au début de chaque nouvelle session de travail pour garantir que le code généré s'intègre parfaitement à l'existant. Rappelez également systématiquement la stack : **React 18 + Vite 5 + TypeScript strict + Tailwind CSS 4 + Shadcn UI + React Query 5 + Express 5 + MySQL (mysql2/promise)**.